

960121

Data Persistence

Session 08



Objective

- In Session 8, we move from the “ephemeral” to a professional **Relational Database**.
- We will learn how to manage the "**Source of Truth**"—the single place where the business knows exactly what happened.
- By using **SQLite**, you get the full power of SQL without the complexity of setting up a heavy server, which is perfect for your individual and group projects.

Relational Logic

To transition from arrays to tables, you need to master the structure of persistent data:

- **Tabular Data vs. Objects:** Understanding why we store data in rows and columns.
- **Primary Keys & Foreign Keys:** The "DNA" of relational logic—how an Order in the `Orders` table "knows" which User in the `Users` table placed it.
- **CRUD Operations in SQL:** Learning that GET/POST/PUT/DELETE in REST maps directly to SELECT/INSERT/UPDATE/DELETE in SQL.

Relational Logic

- **Relational logic** is a powerful subset of *predicate logic* that extends *propositional logic* by using variables and quantifiers to represent relationships between objects, rather than just simple, independent propositions.
- It allows for modeling finite worlds by defining relations (e.g. $p(a,b)$) rather than only checking true/false variables, making it essential for AI, databases, and formal verification.

Differences from Other Logics

- **vs. Propositional Logic:** Relational logic adds variables and quantifiers, allowing for much more compact and expressive representation.
- **vs. Predicate Logic:** It is a form of first-order logic, but often focusing on relational structure (rather than function symbols) and often using finite model theory.

Key Concepts of Relational Logic

- **Atomic Sentences (Atoms):** Formed from n -ary relation constants and n terms (e.g., $p(a,b)$).
- **Variables and Quantifiers:** Uses universal ($\forall$) and existential ($\exists$) quantifiers to express information about multiple objects without enumerating them (e.g. $\forall x. p(x)$).

Key Concepts of Relational Logic

- **Logical Connectives:** Uses standard operators ($\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$) to connect relational sentences.
- **Finiteness Restriction:** Commonly restricted to finite worlds (Herbrand semantics), where there are only finitely many objects to consider, notes the GitHub Pages documentation.
- **Relational Database Mapping:** Often used in databases to define constraints and queries based on first-order logic.

1. The Death of Redundancy (Normalization)

In **Session 4**, students might have saved an order like this in JSON:

```
{ "orderId": 1, "userName": "Alice", "userEmail":  
"alice@email.com", "product": "Laptop" }
```

- **The Problem:** If Alice changes her email, we have to find every single order she ever made and update it. If we miss one, the data is "out of sync."
- **The Relational Logic:** We split the data into two tables.
 - **Users Table:** Stores Alice's name and email once.
 - **Orders Table:** Stores the order but only points to Alice's User_ID.

2. The Connectors: PK and FK

This is the "DNA" of the database.

- **Primary Key (PK):** A unique ID (like a Passport Number) that identifies one specific row in a table. Every table must have one.
- **Foreign Key (FK):** A column in one table (the Child) that points to the Primary Key in another table (the Parent).
- Example for the E-commerce Project:
 - `Users.id` is the PK.
 - `Orders.user_id` is the FK that "borrows" the ID from the Users table to create a link.

2. Visualizing the Tables

Table: Users (The Parent):

This table stores the "who."

ID (PK)	Name	Email
101	Alice	alice@email.com
102	Bob	bob@email.com

2. Visualizing the Tables

Table: Orders (The Child)

This table stores the "what," but it uses the PK from the Users table to identify "who."

Order_ID (PK)	User_ID (FK)	Total_Price	Date
5001	101	\$99.00	2024-03-01
5002	101	\$150.00	2024-03-01
5003	102	\$45.00	2024-03-02

3. Relationship Types in Business

You need to decide how tables "talk" to each other based on business rules:

Relationship	E-commerce Example	Logic
One-to-Many (1:N)	One User has many Orders .	This is the most common. One ID in Table A appears many times in Table B.
Many-to-Many (M:N)	Many Orders contain many Products .	Requires a "Junction Table" (e.g., Order_Items) to sit in the middle.
One-to-One (1:1)	One User has one Profile .	Used for security or performance to keep sensitive data separate.

4. Entity-Relationship Diagram

- An Entity Relationship Diagram (ERD) is a specialized, visual flowchart used in database design to map how data entities—such as people, objects, or concepts—interact within a system.
- Developed by Peter Chen in the 1970s, ERDs use
 - Entities (nouns),
 - Relationships (verbs), and
 - Attributesto structure relational databases, often employing notations like Crow's Foot to show cardinalities.

4. ERD

Key Components of an ERD

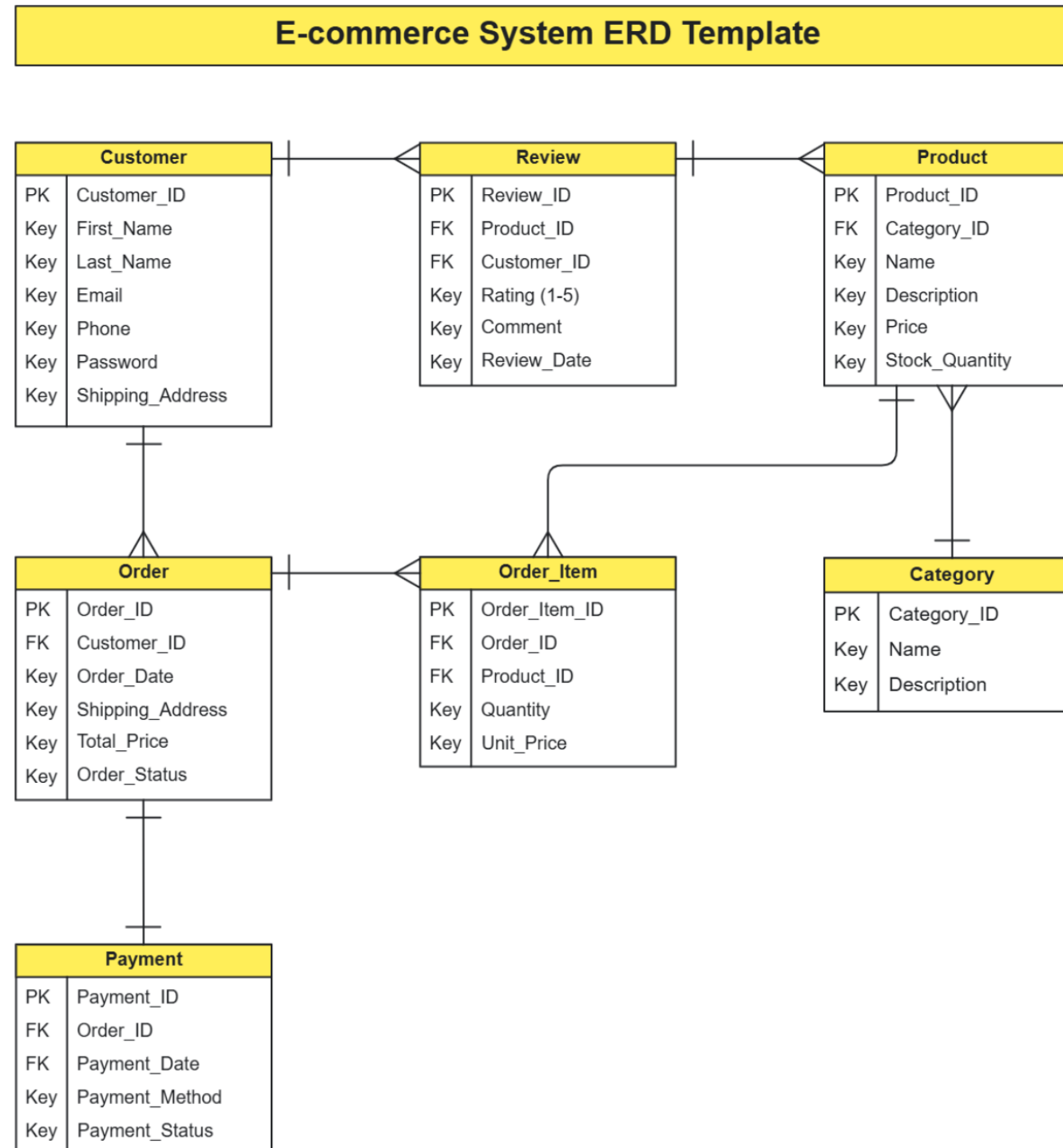
- **Entities**: Represented by rectangles (e.g., Customer, Product), they are the objects about which data is stored.
- **Attributes**: Ovals or text inside the entity box, they define the properties of an entity (e.g., CustomerName, Email).
- **Relationships**: Diamonds or lines, they show how entities interact, such as a Customer "places" an Order.
- **Cardinality**: Defines numerical context, such as one-to-one, one-to-many, or many-to-many (e.g., one customer places *many* orders).

4. ERD

Best Practices for Creating an ERD

- Identify all relevant entities clearly as nouns.
- Ensure every entity has a unique primary key (PK).
- Define relationships using verbs to connect entities.
- Avoid duplicating data and ensure relationships are not redundant. an ERD

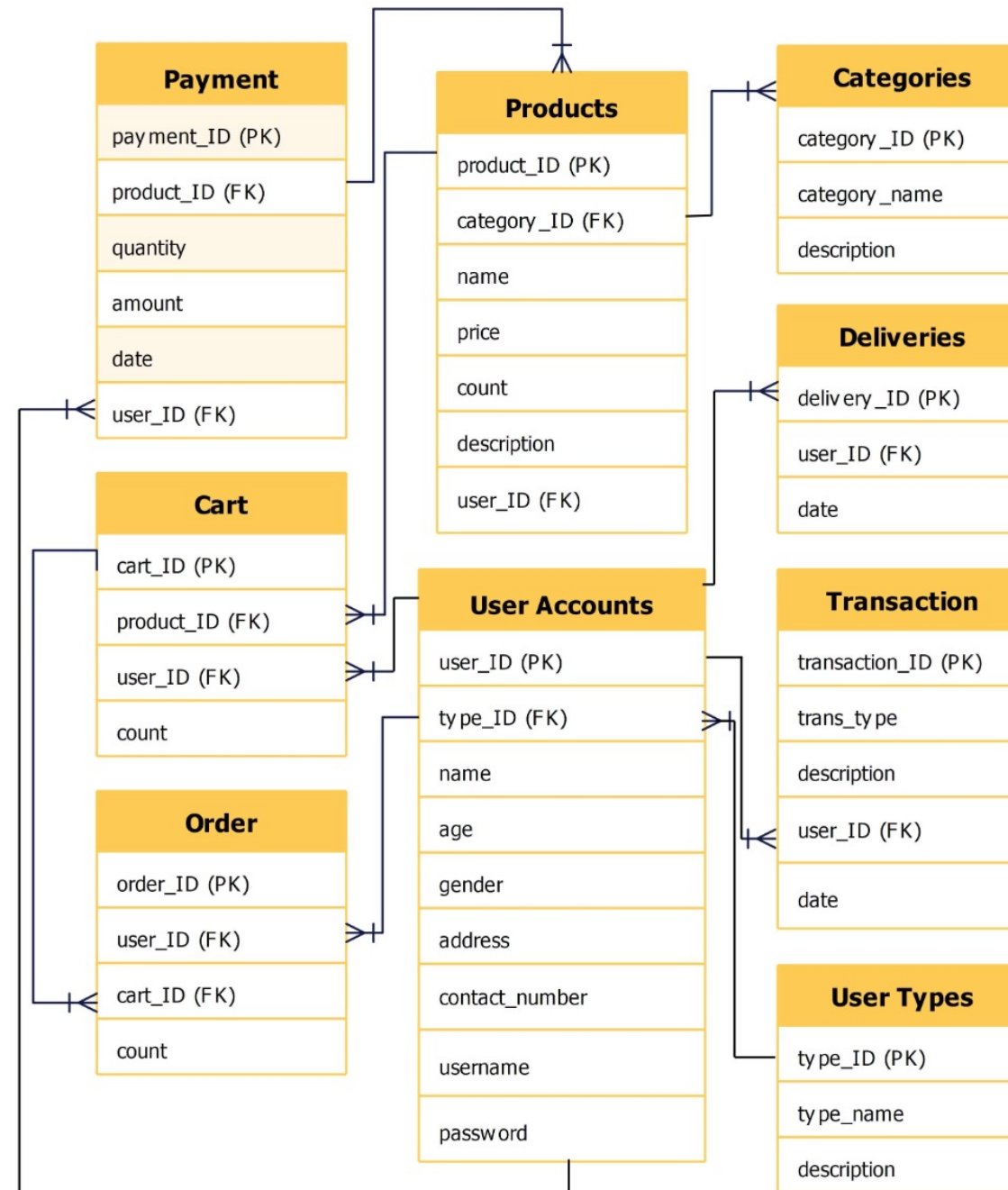
4. ERD



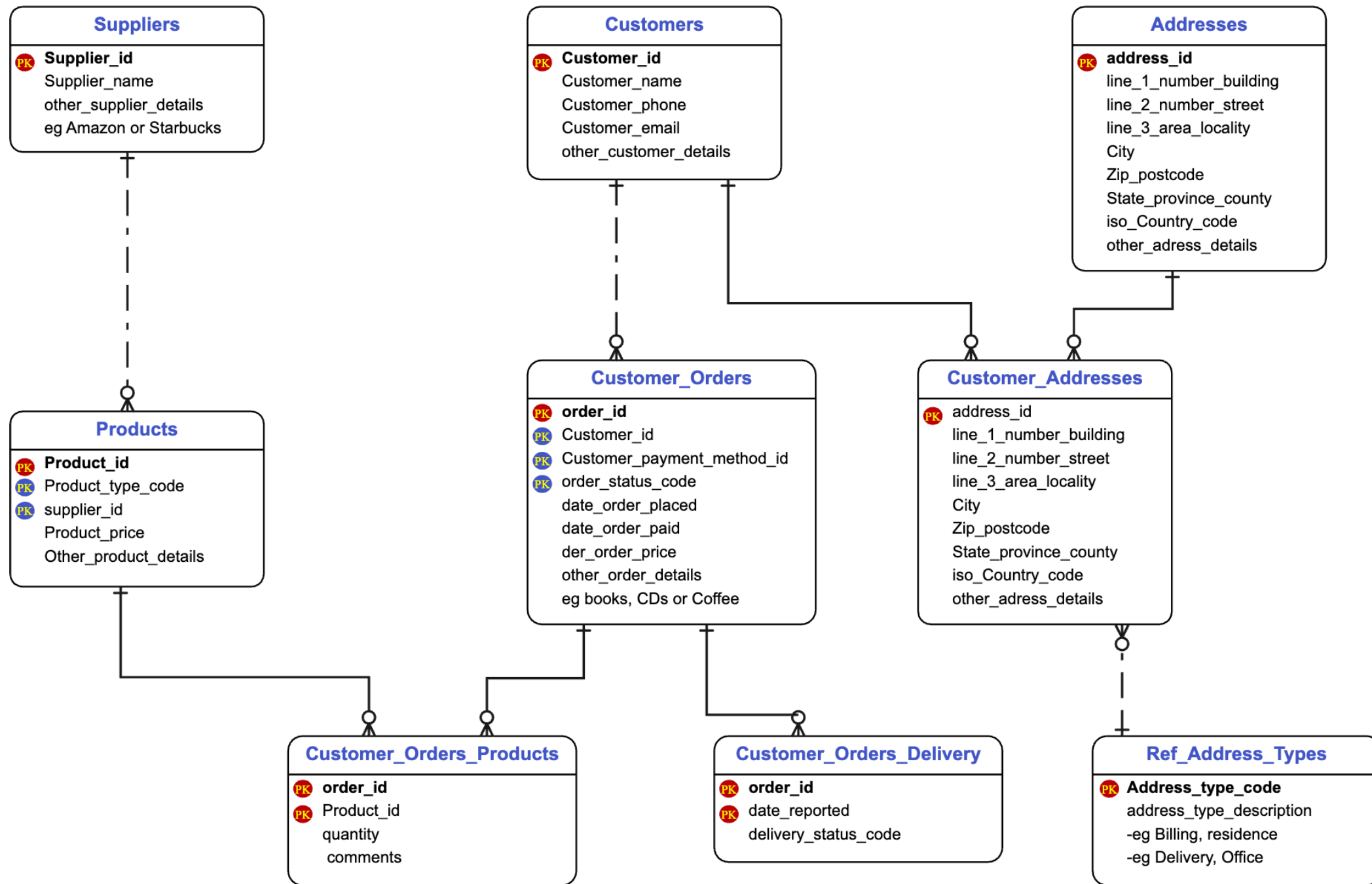
<https://www.template.net/edit-online/498431/e-commerce-system-erd>

ER DIAGRAM : ONLINE SHOPPING CART

4. ERD



<https://edrawmax.wondershare.com/examples/er-diagram-for-online-shopping.html>



5. SQL: The Language of Logic

Explain that SQL is just a way to "Join" these separated tables back together for the UI.

- **Logic:** "Hey Database, give me the `Order_Date` from the Orders table **JOINED** with the `Name` from the Users table where the IDs match."

"Relational Logic is about **Efficiency and Integrity**. We store data in the smallest possible pieces (Tables) and use **Keys** to rebuild the story whenever the user asks for it."

The "Logic Translation" Table

- Since we are moving from "flat" JSON to "structured" SQL, students need to understand that SQL isn't just code—it's a **declaration of logic**.
- When you use GenAI, you shouldn't just ask for "the code to save an order." You should provide the **Relational Logic** so the AI writes a query that maintains data integrity.

The "Logic Translation" Table

Business Goal	Logic Translation	SQL Command (The "Query")
Search: "Find the price of the Laptop."	Find the price in the products table where the name is 'Laptop'.	<code>SELECT price FROM products WHERE name = 'Laptop' ;</code>

The "Logic Translation" Table

Business Goal	Logic Translation	SQL Command (The "Query")
Transaction: "Record a new sale for User 5."	Put a new row into orders with <code>user_id</code> set to 5.	<code>INSERT INTO orders (user_id, total) VALUES (5, 99.99);</code>

The "Logic Translation" Table

Business Goal	Logic Translation	SQL Command (The "Query")
Relation: "Show me Alice's order history."	Get all orders and match their <code>user_id</code> to Alice's id in the users table.	<pre>SELECT * FROM orders JOIN users ON orders.user_id = users.id WHERE users.name = 'Alice';</pre>

The "Architectural" GenAI Prompt

The Prompt Template: Use this specific structure. By defining the **Schema** first, they ensure the AI doesn't hallucinate column names.

"I have an SQLite database with two tables:

- `users (id, name, email)`
- `orders (id, user_id, total_price, status).`

Write a Node.js function using the `sqlite3` library that:

- Validates if a user exists.
- If yes, inserts a new order.
- Uses **parameterized queries** (to prevent SQL Injection).
- Returns the new Order ID."

Critical Thinking: The "SQL Injection" Moment

This is a vital "Gatekeeper" concept for Session 8.

- **The Scenario:** A hacker enters ' OR '1'='1 into the search bar.
- **The Logic:** If you simply "glues" strings together like `SELECT * FROM users WHERE name = ' + userInput`, the hacker can bypass the entire password check.
- **The Fix:** You must use **placeholders** (?).
 - **Wrong Logic:** `db.get("SELECT * FROM users WHERE id = " + req.body.id)`
 - **Right Logic:** `db.get("SELECT * FROM users WHERE id = ?", [req.body.id])`

Referance

- **Introduction to Logic Relational Logic**
(http://intrologic.stanford.edu/lectures/lecture_07.pdf)



Class Activity:

Architecting the Schema

1. The Scenario:

- **The Challenge:** Your server currently saves orders to a JSON file. If two people buy something at the exact same millisecond, the file might get corrupted.
- **Goal:** We will migrate our "Catalog" and "Orders" to an SQLite database to ensure data integrity and allow for complex reporting (like "Show me all orders from last Tuesday").

Architecting the Schema

2. (15 min.) UML:

- Draw an **Entity-Relationship Diagram (ERD)**
- **Logic Check:**
 - You must draw boxes for `Products`, `Users`, and `Orders`.
- **Critical Connection:**
 - You must draw a line between `Users` and `Orders` and label it with `"User_ID"`. This is the first time they visualize **Relational Logic**.

Architecting the Schema

3. (10 min.) **Peer Evaluation:**

- Students swap drawings.
- **Critical Thinking Question:**
 - If I delete a Product from the database, what happens to the historical Orders that contain that product?
 - Should we delete the order, or keep it for accounting?"

Architecting the Schema

4. (5 min.) GenAI Prompt Engineering:

- Students use UML logic to craft a structured prompt.

Prompt:

“Act as a Database Administrator. Write the Node.js code to connect my Express app to a `store.db` file using the `sqlite3` library. Then, write a SQL `INSERT` statement that saves an order consisting of `user_id`, `product_id`, `quantity`, and `total_price`.”

Architecting the Schema

5. (20 min.) **Integration:**

- Install the SQLite driver via npm.
- Open your project in VS Code.
- Replace your `fs.writeFile` (JSON) logic with the new `db.run()` (SQL) logic.

Architecting the Schema

6. (10 min.) Debugging & Testing:

- **The "Direct Access" Test:** Use a VS Code extension (like "SQLite Viewer") to open the .db file directly.
- **Observation:** You will see your "Handwork" ERD manifested as a real table with real data.

The "Add to Cart" Logic

7. (5 min.) Version Control:

- Terminal Command:

- `git add`

- `git commit -m "feat: migrate order storage from JSON to SQLite database."`

- `git push.`

- VS Code:

- GitHub Desktop:

Key Takeaway

- A database is the **memory** of your business. As an architect, your job is to design the **Schema** (the skeleton) so that the memory is organized, searchable, and impossible to corrupt.

Key Takeaway

This session is about **Data Integrity**.

- **Why not JSON?** JSON is a document; SQL is a system. SQL ensures that a `price` is always a number and a `date` is always a date. This prevents "Garbage In, Garbage Out."
- **Relational Power:** Explain that with a database, the business can answer complex questions: *"Which product is our bestseller in the 'Electronics' category?"* This is impossible to do efficiently with just JSON files.
- **Scalability:** Databases are built to handle hundreds of people at once. Files are not.